



---

## **Infrared Finance Incidence Response Security Review**

---

### **Auditors**

0xRajeev, Lead Security Researcher

Noah Marconi, Lead Security Researcher

Optimum, Lead Security Researcher

R0bert, Lead Security Researcher

Chinmay Farkya, Associate Security Researcher

**Report prepared by:** Lucas Goiriz

February 24, 2025

# Contents

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>About Spearbit</b>                    | <b>2</b> |
| <b>2</b> | <b>Introduction</b>                      | <b>2</b> |
| <b>3</b> | <b>Risk classification</b>               | <b>2</b> |
| 3.1      | Impact.....                              | 2        |
| 3.2      | Likelihood.....                          | 2        |
| 3.3      | Action required for severity levels..... | 3        |
| <b>4</b> | <b>Executive Summary</b>                 | <b>4</b> |
| <b>5</b> | <b>Details</b>                           | <b>5</b> |
| 5.1      | Impact.....                              | 5        |
| 5.2      | Root Cause.....                          | 5        |
| 5.3      | Review of Mitigation and Migration.....  | 5        |
| 5.4      | Conclusions and Recommendation.....      | 5        |

# 1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at [spearbit.com](https://spearbit.com)

## 2 Introduction

Infrared simplifies interacting with Proof of Liquidity with liquid staking products such as iBGT and iBERA.

## 3 Risk classification

| Severity level     | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: high   | Critical     | High           | Medium      |
| Likelihood: medium | High         | Medium         | Low         |
| Likelihood: low    | Medium       | Low            | Low         |

### 3.1 Impact

- Critical - an exploit that results in full compromise of the system or network. The attacker gains root/admin- level access. For example (but not limited to) extracting or deleting sensitive data, taking complete control over the infrastructure, or shutting down or severely degrading essential services. *Must* fix as soon as possible (if already deployed).
- High - an exploit that grants unauthorized access to sensitive data or services, potentially causing significant disruption or damage. The attacker does not gain root access but can still perform malicious activities. For example (but not limited to) unauthorized access to confidential data (as defined by the client), ability to modify, delete, or extract non-public data, or perform actions that can degrade user services significantly.
- Medium - a vulnerability that can be exploited to gain unauthorized access to the system, but with limited impact on data or service availability. For example (but not limited to) gaining the ability to bypass certain security controls (e.g., access controls, authentication), or the potential to impact service performance or availability, but not severely.
- Low - losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired.
- Optimizations - Suggestions around design or code improvements that improve performance.
- Informational - Suggestions around best practices or readability.

### 3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

### 3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

## 4 Executive Summary

The Infrared Finance team notified the Spearbit Review team on 13th February over Telegram that they had identified a bug in their production deployment. The bug was supposedly introduced as part of their fix to [Finding 73](#) reported during the Spearbit review conducted from December 16th 2024 - January 10th 2025.

Spearbit review team along with two other researchers (who were part of a private review conducted after the Spearbit review) joined the client team for an incident response review over 13th-14th February to assist in reviewing the codebase, identifying the root cause and evaluating proposed fixes for mitigation of the original bug and any migration issues.

## 5 Details

### 5.1 Impact

1. Infrared users were getting abnormally high rewards if they staked in the same block that `harvestRewards` was called.
2. Six of Infrared Vaults were deemed affected because they were giving out rewards.
3. The excess ~680 iBGT tokens (at communicated time) rewarded would need to be compensated to affected users from ~14K in reserves (at communicated time) coming from collected fees.
4. The root cause would have to be determined and reversed with required mitigation and migration.

### 5.2 Root Cause

1. Instead of skipping only calculations on token ([MultiRewards.sol#L87-L89](#)), the applied fix incorrectly skipped everything in the loop in [MultiRewards.sol#L83-L85](#).
2. Account calculations for that token were also skipped ([MultiRewards.sol#L91-L94](#)), which was the root cause.
3. Scenario triggering the bug: If a user makes a deposit for the first time (by default `userRewardPerTokenPaid[account][_rewardsToken]` will be zero), in a block where `lastUpdateTime` has already been updated, the user's `userRewardPerTokenPaid` might be improperly initialized (or remain zero). Consequently, subsequent reward calculations yield an excessive reward.

### 5.3 Review of Mitigation and Migration

1. The infrared team could have paused a lot of their functionality during investigation but chose not to do so because they did not want to cause panic among users.
2. The Infrared team worked on a mitigation+migration at [recreate-reward-inflation](#) which was reviewed by the Spearbit team in parallel while making recommendations.
3. The mitigation was to revert the earlier applied incorrect fix, shown as struck out code below:

```
modifier updateReward(address account) {
    for (uint256 i; i < rewardTokens.length; i++) {
        address token = rewardTokens[i];

        if (rewardData[token].lastUpdateTime ==
block.timestamp) {
continue;
}

        uint256 latestRewardPerToken =
        rewardPerToken(token);
        rewardData[token].rewardPerTokenStored =
        latestRewardPerToken;
        rewardData[token].lastUpdateTime =
        lastTimeRewardApplicable(token);

        if (account != address(0)) {
            rewards[account][token] = earned(account,
            token);
```

```

                                userRewardPerTokenPaid[account][token] =
latestRewardPerToken;
                                }
                                }
                                -;
                                }

```

4. Most of the remaining changes were related to migration: upgrading the Infrared contract, redeploying vaults and migrating user funds.
5. Upgrade - changes from **Infrared** to **InfraredV2**:
  - a. Add **migrateVault** function (**onlyGov**) - pauses staking in old vault, deploys new vault, switches vault in registry.
  - b. Remove **initialize** as proxy has already been initialized.
  - c. Remove **setIBGT** as **ibgt** is already set.
  - d. Add **initializeV2** for auto migrating 6 active reward vaults.
  - e. Add **recoverERC20FromOldVault** to recover any stray ERC20 rewards from old vaults once registry switches.
  - f. Add **harvestOldVault** as **onlyGov** to harvest from old vaults with stakes remaining
6. Users would be prompted to migrate their funds themselves with assistance from Infrared front-end which would batch these 3 calls together:
 

```

oldvault.exit();
LP.approve(newvault, amount);
newvault.stake(amount);

```
7. We recommended restricting repeated calls to **initializeV2** using **reinitializer**. Fix: **onlyGovernor** was added.
8. We recommended using a version string to allow potentially more migrations if required in future. Fix: **versionToUpgradeTo** was added in **function migrateVault(address \_asset, uint8 versionToUpgradeTo)**
9. We recommended deploying a helper contract to revert if a withdraw / exit occurs in the same block as an earlier transaction that would trigger **updateReward** to prevent loss of rewards. The project deployed a helper and encouraged use of it through the UI for stakers migrating.
10. The project intends to collect stuck rewards, if any, from the old vaults by removing the reward token and using **recoverERC20**. Any recovered rewards would be notified to the new vaults. It is important to ensure that once a reward token is removed it is not added again as staker level accounting is not reset.

Various other scenarios were discussed to understand and prevent any unexpected side-effects from the mitigation and potential exploit scenarios during/after the migration.

## 5.4 Conclusions and Recommendation

Spearbit team performed a best-effort spot review of the mitigation and migration changes initiated by the Infrared team over the duration of two days. To the best of our knowledge, these changes seemed reasonable in this specific context. Changes made to the codebase between the time of our initial review and this incident response review (as a result of findings/recommendations from the private review, Cantina contest or others) were outside our scope. The focussed scope/duration of this incident response review was not sufficient to evaluate those other changes or their side-effects to the recommendations given here.

We strongly recommend increasing the level of test coverage and potentially another review to gain a higher level of assurance and reduce any latent risk. We strongly recommend not deploying any code/fixes that have not been externally reviewed. We also recommend creating a bug bounty program if not done already.